

Technical Interview Evaluation

CANDIDATE NAME	DATE	INTERVIEWER

LEVEL 01 · JUNIOR

Foundations

1.1 *Pass by Value vs. Pass by Reference*

What is the difference between the two at the memory level?
What kind of race-condition bugs typically arise in this context?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

1.2 *Database Index*

When does an index speed the system up, and when does it slow it down?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

1.3 *Normalization vs. Denormalization*

Explain both concepts at the database level.
Why do teams sometimes intentionally denormalize data?
Which approach is more common in NoSQL databases?
Which approach is more common in SQL databases?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

2.1 *CAP Theorem*

What is the CAP theorem, and why can't all three properties be guaranteed simultaneously in a distributed system?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

2.2 *Synchronous vs. Asynchronous*

Imagine a system that must call ten external APIs — why is async / non-blocking usually better?
Is async always faster? When is sync the better choice?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

2.3 *Stateless vs. Stateful*

Suppose you've designed two systems, one stateless and one stateful — what are the trade-offs of each?
Why is REST typically designed to be stateless?
Where would you store session data in a stateless system?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

3.1 *Polling vs. Webhook vs. Long Polling vs. WebSocket*

You're building a notification system — which would you choose, and why?

How do latency and server load differ across these options?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

3.2 *Idempotency*

You're designing a payment endpoint *POST /payments* handling millions of requests per day. Clients (mobile, web, services) may resend a request for many reasons — how do you handle it?

Who should generate the idempotency key — client or server? Why?

If the client generates it, what risks exist (e.g. collision, abuse)?

If the server generates it, how can it detect that a request is a duplicate?

SCORE 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐

1 = weak · 5 = excellent

Final Summary

STRENGTHS

WEAKNESSES

RECOMMENDED LEVEL

☐ Junior

☐ Mid

☐ Senior

☐ Staff / Lead

☐ Not a fit

FINAL DECISION

☐ Strong Hire

☐ Hire

☐ On the Fence

☐ No Hire

NOTES

INTERVIEWER SIGNATURE

DATE